A Backpack Full of Skills: Egocentric Video Understanding with Diverse Task Perspectives

Simone Alberto Peirone[1] Francesca Pistilli[1] Antonio Alliegro[1,2] Giuseppe Averta[1]
[1] Politecnico di Torino, [2] Istituto Italiano di Tecnologia
firstname.lastname@polito.it

Abstract

Human comprehension of a video stream is naturally broad: in a few instants, we are able to understand what is happening, the relevance and relationship of objects, and forecast what will follow in the near future, everything all at once. We believe that - to effectively transfer such an holistic perception to intelligent machines - an important role is played by learning to correlate concepts and to abstract knowledge coming from different tasks, to synergistically exploit them when learning novel skills. To accomplish this, we seek for a unified approach to video understanding which combines shared temporal modelling of human actions with minimal overhead, to support multiple downstream tasks and enable cooperation when learning novel skills. We then propose EgoPack, a solution that creates a collection of task perspectives that can be carried across downstream tasks and used as a potential source of additional insights, as a backpack of skills that a robot can carry around and use when needed. We demonstrate the effectiveness and efficiency of our approach on four Ego4D benchmarks, outperforming current state-of-the-art methods. Project webpage: sapeirone.github.io/EgoPack.

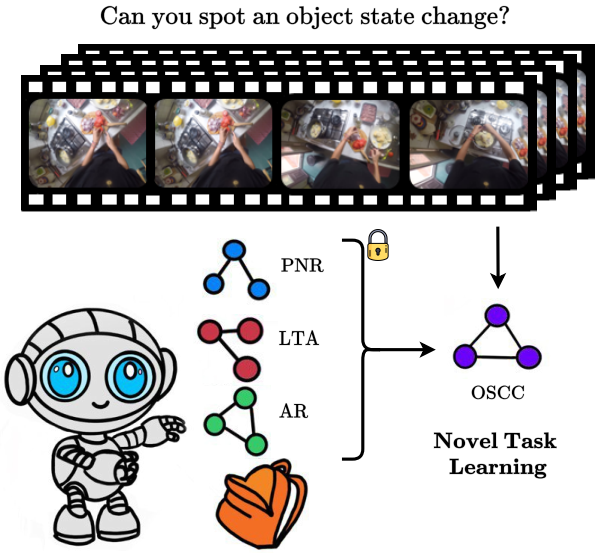


Figure 1. Given a video stream, a robot is asked to learn a novel task, *e.g.* Object State Change Classification (OSCC). To learn the new skill, the robot can access previously gained knowledge regarding different tasks, such as Point of No Return (PNR), Long Term Anticipation (LTA) and Action Recognition (AR), and use it during the learning process to enhance downstream task performance. This knowledge is stored as graphs inside the robot's backpack, always ready to boost a new skill.

1. Introduction

Our daily living activities are extremely complex and diverse, nonetheless humans have the extraordinary ability to reason on the behaviour itself in just a few instants from a visual input. We are able to spot what another person is doing, predict their next actions based on current observations, and understand the implications of an activity, for instance whether its effects are reversible. Observing someone in the kitchen by the worktable, where there is a pack of flour and a jug of water, we can identify that they are a chef kneading flour (reasoning about current activity). We can also forecast that the next step will involve mixing the flour with water (reasoning about the future), and finally obtaining dough (reasoning about implications of these actions). This type of holistic reasoning, which is natural for humans, is still a distant goal for artificial intelligence systems. The challenge arises not only from the requirement of executing multiple tasks with a single architecture, but also from the necessity of being able to abstract and repurpose such knowledge across-tasks, for example to foster and enhance the learning of novel skills. Current research trends in human activity understanding predominantly focus on creating several, hyper-specialised, models. This approach splits the understanding of human activities into distinct skills, with each model being independently trained to rely only on "task-specific" clues for prediction [69, 73, 76]. However, this approach disregards the valuable insights that could be gleaned from different task perspectives. A first step in this direction relies on Multi-Task Learning (MTL) to exploit

the intuition that knowledge sharing between tasks may improve performance. However, the proposed multi-task models have some limitations [40], mostly concerning a negative transfer between tasks, making it difficult to outperform single-task models. Most importantly, MTL usually assumes the availability of supervision for all tasks at training time, limiting the extension of the models at a later time.

The recently proposed EgoT2 framework [68] offers a unified solution to integrate various egocentric video tasks. It employs an ensemble of diverse, task-specific models and learns to translate task-specific clues through a transformer-based encoder-decoder to benefit one of the tasks. Although this approach fosters positive interactions between tasks, it has significant limitations: i) the primary task should be "known" at training time and present within the task-specific models collection, ii) it necessitates an extensive pretraining process and iii) it lacks a knowledge abstraction, as it relies on task-specific models rather than creating transferable concepts.

Indeed, we argue that an important key to advance the learning capabilities of intelligent systems and to move a step closer to a generalised reasoning on visual understanding involves not only sharing information across tasks, but also abstracting task-specific knowledge for application in new scenarios. Considering an ensemble of vision tasks, each offers a distinct perspective on the input stream and extracts different types of information. Our goal is to encapsulate this diverse knowledge to be leveraged in the future to positively impact the learning of a novel skill. We focus on egocentric video understanding as it is the perfect harbour to study human activities and synergies between tasks. There is a strong connection between egocentric tasks. For instance, specific actions, like peeling a potato, directly result in a change in the state of the object (the potato in this case), illustrating the interconnected nature of these tasks.

All the above considerations motivate us in investigating new alternatives and we propose a novel framework for knowledge abstraction and sharing called EgoPack. Our underlying idea, is to exploit a set of known tasks, each one able to interpret an input stream according to its own perspective, to learn reusable knowledge that can aid the learning process of a novel task. We show this concept in Fig. 1, where a robot is equipped with a backpack that figuratively summarises all the knowledge gained from a set of tasks. To learn a new skill, the robot can "take-out" task-related knowledge from the backpack and leverage it within the learning process. The task-specific perspectives are collected in a single pretraining step of a novel multi-task network under the form of prototypes. We exploit a new versatile temporal graph-based architecture shared across all the tasks, with minimal overhead to support each task.

When learning a new skill, EgoPack promotes the interaction between the different tasks by learning which relevant knowledge to extract from the different perspectives. The architecture of EgoPack is notably flexible, enabling easy adaptation to novel tasks by reusing the previous tasks to facilitate the learning process of any novel task.

We demonstrate the effectiveness and efficiency of our approach on Ego4D [25], a large-scale egocentric vision dataset. To summarise, our main contributions are:

1. We present a unified architecture to learn multiple egocentric vision tasks with minimal task-specific overhead;
2. We introduce EgoPack, a novel approach that leverages different task perspectives to build a robust knowledge abstraction which can foster the learning of a novel task;
3. Our approach outperforms both specialised single and multi-task baselines by leveraging the unique synergies and distinct perspectives of different tasks;
4. EgoPack achieves competitive performance on Ego4D [25] for all the considered benchmarks, outperforming the state-of-the-art on some.

2. Related Works

Egocentric Vision Egocentric vision captures human activities from the privileged perspective of the camera wearer, allowing a unique point of view on the actions [3, 49]. Recently, the field has seen rapid development thanks to the release of several large-scale egocentric vision datasets [11, 12, 25, 30, 34, 56]. The rich annotations of these datasets [12, 25] allow to tackle a large number of tasks, including action recognition [46], action anticipation [18, 23, 76], next active object prediction [19], action segmentation [33, 73] and episodic memory [52]. Previous works in egocentric vision have focused on domain adaptation [6, 44, 48, 50, 70], multimodal learning [20, 62, 70] and large-scale video-language pretraining [1, 51, 75] to learn better representation for downstream tasks.

Graph Neural Networks for vision tasks Traditional neural networks, including Convolutional Neural Networks (CNNs), have been widely used in computer vision, showing impressive performance on a variety of problems [26, 39, 41]. However, these models often assume data lying on a regular domain, such as images that have a grid-like structure. In recent years, the interest in developing methods able to provide a more general and powerful type of processing has been growing and particular attention has been given to learning methods on graphs. Graph Neural Networks (GNNs) have the innate ability to effectively handle data that lie on irregular domains, such as 3D data [58, 66], robotics [47], molecular chemistry [37], and social or financial networks [15], and to model complex data relations [55]. Recently, transformer-based architectures had a great impact on vision application. Despite Transformers and GNNs share some similarities in their ability to handle various data types, they are fundamentally different in their

core architectures and the specific ways they process data. GNNs can model the topology of a graph and the relations between nodes while also inheriting all the desirable properties of classic convolutions: locality, hierarchical structures and efficient weight reuse. In video understanding GNNs have been applied to action localisation [22, 33, 53, 72], to build a knowledge graph from human actions [21], to model human-object interactions [13, 14] or to build a topological map of the environment [45].

Multi-Task Learning MTL [5, 74] tackles the problem of learning to solve multiple task simultaneously. The development of this strategy is justified by the intuition that complex settings require solving multiple tasks, for instance autonomous driving [31], robotics and natural language processing. Furthermore, these networks can bring the theoretical advantage of sharing complementary information to improve performance. Several works have been done in this direction [7, 8, 10, 17, 31, 32, 40, 57], focusing on which parameters or tasks is better to share [28, 35, 60, 61] and promoting synergies between tasks [36, 65]. Such methods encounter the problem of negative transfer [40] and sharing with unrelated tasks [28, 60] consequently suffering of task competition and not being able to benefit from information sharing between tasks. To overcome these limitations, several methods have been proposed to balance task-related losses [9, 38, 59], to dynamically prioritise tasks [27], to reduce gradient interference between tasks [71] or to exploit task interactions at multiple scales [63]. Unfortunately, all these solutions require extensive task-specific tuning, and are not able to build an holistic perception across tasks. Few works have explored MTL in the field of egocentric vision [32, 36, 65, 68]. Among these, the recently proposed EgoT2 [68] builds an ensemble of diverse, task-specific models. The features of the different models are projected into a common feature space and processed through a transformer-based encoder-decoder to translate the contributions of different tasks and generate predictions for the primary task. Notably, the primary task has to be part of the task-specific models. This approach fosters positive interactions between tasks, resulting in improved performance compared to the single-task models. However, it has some limitations, as it is not able to build knowledge abstractions that can be easily transferred to novel tasks. Instead, we propose a model that can build a robust backpack of task perspectives that can be used in learning any novel tasks.

3. Method

We tackle a task cooperation setting, in which an egocentric vision model is able to exploit previously acquired knowledge over task perspectives to foster the learning process of any novel task. We formulate the proposed setting in Sec. 3.1. We present a unified temporal architecture to model multiple tasks in Sec. 3.2, a key step to enable knowledge sharing between tasks. Finally, Sec. 3.3 presents our novel approach EgoPack to enable efficient transfer of different task perspectives to novel tasks.

3.1. Setting

A task $\mathcal{T}$ is associated with a dataset $\mathcal{D} = \{(v_i, y_i)\}_{i=1}^{N}$, where v_i is a video segment of arbitrary length, y_i is the associated ground truth label and N is the number of segments. Our approach follows a two-stages training process. First, a model $\mathcal{M}$ is trained on a set of K tasks $\{\mathcal{T}_0, \dots, \mathcal{T}_K\}$, under a Multi-Task Learning framework with hard-parameter sharing [54] to encourage the model to learn more general and task-agnostic representations thanks to the joint supervision of multiple tasks. Then, the model is presented with a new task $\mathcal{T}_{K+1}$ to learn, without access to the supervision of the previous tasks. In this scenario, the new task may benefit from potential semantic affinities with the previously seen tasks. For example, a model that has learnt to detect object changes may apply this knowledge for action recognition and vice-versa, as some actions are associated with object changes, *e.g. cutting something*, while others are not, *e.g. moving an object*. Our goal is to make these semantic affinities more explicit (and exploitable) so that the new task can learn to repurpose these *perspectives* from previous tasks to improve performance, a step towards more holistic models that seamlessly share knowledge between tasks.

3.2. A Unified Architecture for Egocentric Tasks

The main premise of our method is that different egocentric vision tasks can be modelled using a shared architecture with minimal differences between tasks. Under this assumption, videos can be seen as a sequence of N temporal segments encoded as $\mathbf{x} = \{\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_N\}$, where $\mathbf{x}_i \in \mathbb{R}^D$ represents the D-dimensional features of segment v_i extracted using some video features extractor $\mathcal{F}$, *e.g.* SlowFast [16] or Omnivore [24]. Such sequence could be interpreted as a temporal graph $\mathcal{G}(\mathcal{X}, \mathcal{E})$, whose nodes $\mathbf{x}_i \in \mathcal{X}$ represent the segments of the video, and edges $e_{ij} \in \mathcal{E}$ connect nodes $\mathbf{x}_i$ and $\mathbf{x}_j$ with a temporal distance considered relevant when lower than a threshold τ. The connectivity of the graph defines the extent of its temporal modelling, *i.e.* connecting further apart nodes enables longer range temporal understanding which could benefit for example anticipation tasks. The threshold τ depends on the task at hand and more implementation details are provided in Sec. 4.1. The temporal position of each node in the sequence is encoded by adding to the node embeddings a positional encoding [64].

This formulation enables the use of Graph Neural Networks (GNNs) to learn the complex temporal relations between video segments and to cast different egocentric vision

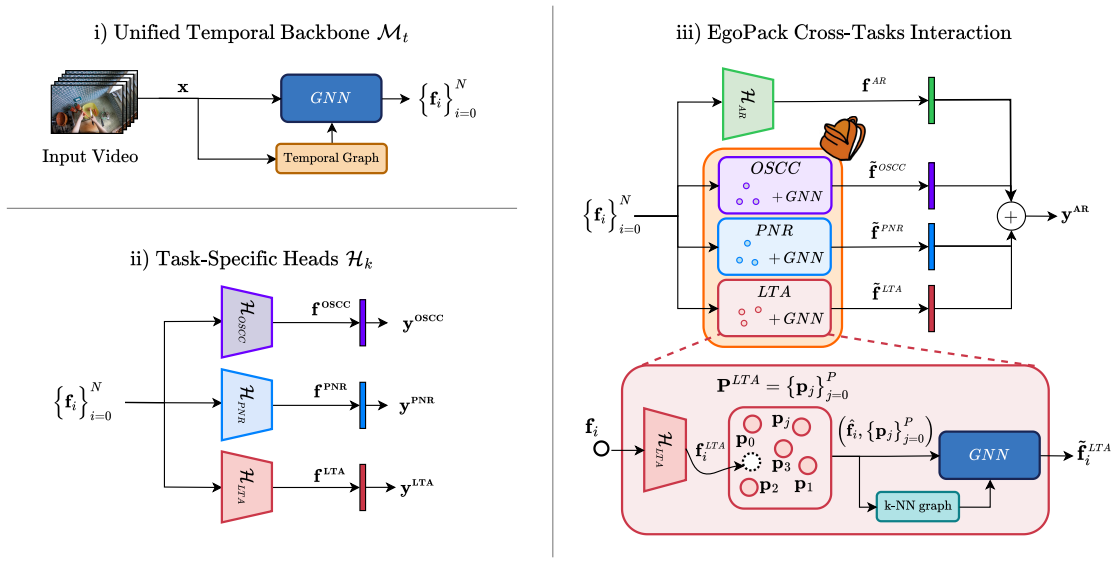


Figure 2. Architecture of EgoPack when Action Recognition (AR) is the novel task. Videos are interpreted as a graph, whose nodes $\mathbf{x}_i$ represent actions, encoded as features, and edges connect temporally close segments. This representation enables the design of a *Unified Temporal Backbone* to learn multiple tasks with a shared architecture and minimal *Task-Specific Heads*, leveraging GNNs for temporal modelling. We exploit this architecture to jointly learn K tasks, *e.g.* OSCC, LTA and PNR. After this training process, we extract a set of prototypes $\mathbf{P}^k$ that summarise what the network has learnt from each task $\mathcal{T}_k$, like a backpack of skills that we can carry over. In this *Cross-Tasks Interaction* phase, the network can peek at these different task-perspective to enrich the learning of the novel task.

tasks as graph prediction tasks, such as node-level or graph-level classification, as shown in Fig. 3. This assumption is reflected in our approach by decomposing the multi-task model $\mathcal{M}$ into two components: a general *temporal* backbone $\mathcal{M}_t : \mathbb{R}^D \mapsto \mathbb{R}^{D_t}$, and a set of task-specific projection heads $\mathcal{H}_k : \mathbb{R}^{D_t} \mapsto \mathbb{R}^{D_k}$ mapping the graph and/or the nodes to the desired output space for task $\mathcal{T}_k$ with dimension D_k, as shown in Fig. 2. $\mathcal{M}_t$ is a GNN with L layers that takes as input the temporal sequence $\mathbf{x}$ and provides as output the updated feature vectors $\mathbf{f} = \{\mathbf{f}_1, \mathbf{f}_2, \dots, \mathbf{f}_N\}$. At layer l, node embeddings are projected and combined with their neighbours, following the GraphSAGE architecture [29]:

$$\mathbf{f}_i^{(l+1)} = \mathbf{W}_r^{(l)}\mathbf{f}_i^{(l)} + \mathbf{W}^{(l)} \cdot \mathbf{g}_i^{(l+1)} + \mathbf{b}^{(l)}, \tag{1}$$

where $\mathbf{f}_i^{(l)} \in \mathbb{R}^{D_t^{(l)}}$ are the features of node $\mathbf{x}_i$, $\mathbf{b}^{(l)} \in \mathbb{R}^{D_t^{(l+1)}}$ is a bias term, $\mathbf{W}_r^{(l)}, \mathbf{W}^{(l)} \in \mathbb{R}^{D_t^{(l+1)} \times D_t^{(l)}}$ are the weight matrices associated to the root node and the aggregated neighbours' contribution $\mathbf{g}_i^{(l+1)}$ respectively. The latter is computed as:

$$\mathbf{g}_i^{(l+1)} = \underset{\mathbf{f}_j \in \mathcal{N}_i}{mean}\Big(\phi\big(\mathbf{W}_p^{(l)}\mathbf{f}_j^{(l)} + \mathbf{b}_p^{(l)}\big)\Big), \tag{2}$$

where $\mathbf{W}_p^{(l)} \in \mathbb{R}^{D_t^{(l)} \times D_t^{(l)}}$ projects the neighbours before the aggregation step, ϕ is a non-linearity, $\mathbf{b}_p^{(l)} \in \mathbb{R}^{D_t^{(l+1)}}$ is a bias term and $\mathcal{N}_i$ is the set of neighbours of node $\mathbf{x}_i$. Each layer is followed by Layer Normalization [2] and a LeakyReLU activation function. A residual connection around the temporal GNN allows the network to preserve the original features. Intuitively, the neighbourhood $\mathcal{N}_i$ reflects the temporal dependencies of the input sequence and the GNN allows to iteratively extend the temporal receptive field of each node.

Task-specific heads The output features of the temporal backbone $\mathcal{M}_t$ are shared across the different downstream tasks. To project these features into task-specific components, we add a set of projection heads $\mathcal{H}_k$, one for each task $\mathcal{T}_k$. For graph classification tasks, the nodes of each graph are aggregated using max pooling to obtain a unique features representation. In each head, a MultiLayer Perceptron outputs the task-specific features $\mathbf{f}_i^k \in \mathbb{R}^{D^k}$ and is followed by a linear layer to compute the task logits $\mathbf{y}_i^k \in \mathbb{R}^{D_o^k}$, where D_o^k is the number of labels for task $\mathcal{T}_k$. By limiting the task-specific portion of the network to the heads while sharing the temporal backbone, we can obtain the perspective of all tasks with a single forward through the latter. The network is trained on all the tasks by averaging their losses.

3.3. Learning a novel task with a backpack

To solve the new task, the naive approach would be to finetune the model, adding a new head $\mathcal{H}_{K+1}$ and possibly updating the temporal backbone $\mathcal{M}_t$. However, finetuning may not fully leverage the insights from other tasks as it could result in the loss of the previously acquired knowledge, as confirmed experimentally in Sec. 4.2.

Building the backpack We propose to explicitly model the perspectives of the different tasks as a set of task-

specific prototypes that abstract the knowledge gained by the previously seen tasks and can be accessed by novel tasks. We call this approach EgoPack and provide an overview in Fig. 2. These task-specific prototypes are collected from videos annotated for action recognition, as human actions can be seen as the common thread behind the different tasks. Practically, we forward these samples through the temporal backbone and take the output of the different task-specific projection heads, thus encoding the perspective of each task given the same input video. Finally, the features obtained from each task are aggregated according to the *verb* and *noun* labels of the action, effectively summarising the perspective of each task given the same input action. The result is a set of prototypes $\mathbf{P}^k = \{\mathbf{p}_0^k, \mathbf{p}_2^k, \dots, \mathbf{p}_P^k\} \in \mathbb{R}^{P \times D_k}$ for each task $\mathcal{T}_k$, where P is the number of unique *(verb, noun)* pairs in the dataset and D_k is the size of the task-specific features. These prototypes are frozen and represent a "summary" of what the models has learnt during the multi-task pretraining process, creating an abstraction of the gained knowledge. They can be then reused when learning novel tasks, like a backpack of skills that the model can carry over.

Leveraging the backpack During the learning process of the novel task $\mathcal{T}_{K+1}$, the model can exploit the task prototypes obtained via the task-specific heads. As before, the output of the temporal backbone $\mathbf{f}_i$ is forwarded through all the projection heads to obtain the task-specific features $\mathbf{f}_i^k$. These features are used as *queries* to match the corresponding task prototypes $\mathbf{P}^k$, selecting the k-Nearest Neighbours among the prototypes using cosine similarity in the features space. Task features and their neighbouring prototypes form a *graph-like* structure, on which message passing can be used to enrich the task-specific features $\mathbf{f}_i^k$, following an iterative refinement approach. In particular, at each layer l we select the closest prototypes with k-NN and update the features $\mathbf{f}_i^{(l),k}$ according to the following rule:

$$\mathbf{f}_i^{(l+1),k} = \mathbf{W}_r^{(l)} \mathbf{f}_i^{(l),k} + \mathbf{W}^{(l)} \cdot \max_{\mathbf{p}_j^k \in \mathcal{N}_i^{(l),k}} \mathbf{p}_j^k, \tag{3}$$

where $\mathbf{p}_j^k \in \mathcal{N}_i^{(l),k}$ are the closest prototypes in $\mathbf{P}^k$ to $\mathbf{f}_i^{(l),k}$ and $\mathbf{W}_r^{(l)}, \mathbf{W}^{(l)} \in \mathbb{R}^{D^k \times D^k}$ are the weight matrices associated to the input features and the aggregated neighbours respectively. Notably, only the task features are updated while the task prototypes remain frozen to preserve the original perspectives seen by the network.

In this process, the task-specific heads $\mathcal{H}_k$ are initialised from the multi-task training and possibly updated during the task-specific finetuning process, allowing the model to freely explore the set of task prototypes and to select the most informative ones for each input sample. After the interaction phase, the refined features $\tilde{\mathbf{f}}_i^k$ are fed to a classifier module to obtain the task logits $\mathbf{y}_i^k \in \mathbb{R}^{D_o^k}$ for each task $\mathcal{T}_k$ in the backpack. The final prediction is the sum of the pre-softmax logits coming from the different tasks and the output of a new head $\mathcal{H}_{K+1}$ for the novel task. Intuitively, we allow each task to cast a vote on the final prediction, based on its perspective on the same video segment. In this phase, the temporal network, the task-specific heads and the weights of the GNNs are trained jointly using only the supervision of the novel task $\mathcal{T}_{K+1}$.

4. Experiments

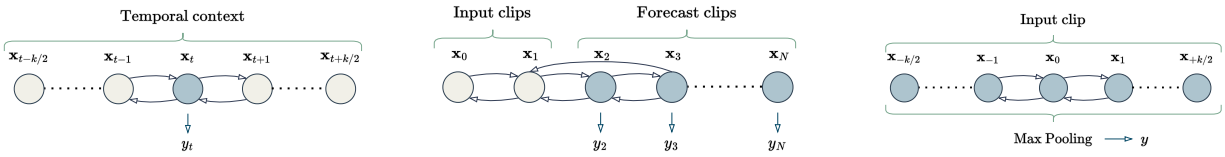
We evaluate EgoPack on four Ego4d Human-Object Interaction benchmarks: Action Recognition (AR)[1], Long Term Action Anticipation (LTA), Object State Change Classification (OSCC) and Point Of No Return (PNR). We report verb and noun top-1 accuracy for AR, accuracy for OSCC, edit distance for LTA and temporal localisation error (in seconds) for PNR.

4.1. Implementation Details

We use Omnivore Swin-L [24] features pre-trained on Kinetics-400 [4], released as part of Ego4D [25] and extracted using dense sampling over a window of 32 frames with a stride of 16 frames and features size 1536. In principle, EgoPack is agnostic to the underlying features extractor and could adopt other architectures. Following previous works on Ego4D [50] we use TRN [77] to temporally aggregate features from the three sub-segments of each input sample. The mapping between videos of each task and its corresponding temporal graph is task dependent, as shown in Fig. 3:

- **Action Recognition (AR)**: actions are mapped to the nodes of the temporal graph $\mathcal{G}$, and edges connect each node to the previous and next (Fig. 3a). To account for the variable length of videos, actions are processed in fixed size windows.
- **Long Term Anticipation (LTA)**: each input clip is mapped to a node in $\mathcal{G}$. Then, a sequence of new nodes is added to the graph, equivalent in number to the clips to forecast. These nodes are initialised with the mean features of the input clips and are connected to the previous and subsequent nodes in the sequence, as well as to the input clips (Fig. 3b).
- **Object State Change Classification (OSCC) and Point of No Return (PNR)**: each input segment is further split into n sub-segments to account for the finer temporal granularity required by these tasks. Each sub-segment is mapped to a node in $\mathcal{G}$, and edges connect each node to the previous and next (Fig. 3c).

[1] AR is not an official Ego4D benchmark and was derived from the LTA annotations by [68]. To be consistent with previous works, we use the v1 version of the LTA annotations.



(a) Node classification (AR, PNR) (b) Future node classification (LTA) (c) Graph classification (OSCC)

Figure 3. Egocentric vision tasks as graph prediction tasks. In AR and LTA, each node is an action within a temporal sequence and the objective is to predict the verb and noun labels of the nodes. In OSCC and PNR, nodes represent different temporal segments of the video clip and the goal is to output a global prediction for the entire graph (OSCC) or the individual nodes (PNR).

Tasks have different annotations and are modelled as separate graphs, even though the temporal model is shared. The task prototypes are built using samples from the train split of the AR dataset. Tasks are trained with standard cross entropy loss, with the exception of PNR which uses binary cross entropy. EgoPack is trained for 30, 40 and 10 epochs for AR, LTA and OSCC/PNR respectively, with a learning rate of $1e-4$ and a linear warm-up for the first 5 epochs, using Adam optimiser and batch size 16. All tasks share the same temporal and cross-task interaction architecture, with minimal task-specific hyper-parameter tuning. More details are reported in the supplementary.

4.2. Quantitative results

We show the main results of EgoPack in Table 1. To assess the validity of our approach, we proceed incrementally starting from single tasks models, *i.e.* each task is trained separately. In this setting, we compare a simple MLP baseline trained on the temporally aggregated features against our temporal graph methodology, which exhibits superior average performance. The improvement is particularly evident in the PNR task, *e.g.* from $1.76\,s$ to $0.61\,s$, where the subpar outcomes of the MLP can be attributed to the lack of explicit temporal modelling. In addition to higher performance, the temporal graph enables all the tasks to be modelled using a unified architecture which allows to train all the tasks at the same time (MTL). With the MTL model, we observe a significant drop in average performance, mostly driven by worse accuracy in AR and OSCC. This behaviour is the result of negative transfers between tasks when they are trained together [67].

Cross-Task Interactions We compare our approach EgoPack for efficient cross task interaction with EgoT2 [68], which learns to combine multiple task-specific frozen models to solve one of them. Unlike EgoPack, the learning process of EgoT2 is divided in two stages, *i.e.* a pre-training step where each individual task is learned from scratch and a task-specific translation step, where just one task of the collection is fine-tuned. Notably, both steps require the supervision of the downstream task. On the contrary, the multi-task pre-training of EgoPack is agnostic to the novel downstream task, potentially allowing to transfer the gained knowledge to any new task. To ensure a fair comparison with EgoPack, we re-implemented the task translation mechanism proposed by EgoT2 on top of our *Temporal Graph* single task models using Omnivore features. This approach is indicated as *Task Translation* in Table 1. Additional details on its implementation are provided in the supplementary. One of the main benefit of our approach is that it requires a single forward pass through the features extraction and temporal backbones to obtain the perspectives of the different tasks, unlike EgoT2 which requires a forward pass for each single task model. Notably, we also highlight that EgoPack obtains better or comparable performance even though the backbone used for features extraction was not trained on Ego4D.

Ablation of the different contributions We summarise the main steps leading to EgoPack in Table 2, using an aggregated metric to capture the overall improvement across the various tasks when compared to the baseline. The metric is computed as an average of the individual task metrics. We adjusted the metrics by taking one minus the score for LTA and PNR, as lower values are preferable, and clipped the PNR localisation error at 1.0 to have the same scale across all the metrics. Temporal modelling alone greatly improves the score compared to the baseline. Although MTL allows to train under a multi-task objective, it clearly underperforms the temporal model due to negative transfers [40]. *Task Translation* partially recovers this gap on some tasks as shown in Table 1, but overall the aggregated metric is comparable with MTL. We speculate that the marginal improvement of *Task Translation* compared to MTL lies in the limited task-specific context the former has access to, as it can peek at the different perspectives of the auxiliary tasks only for the input video at hand, rather than looking at the entire knowledge gained by the model. On the contrary, the task prototypes of EgoPack allow to carry a more complete summary of what the models has learnt from which it can extract useful knowledge based on the sample and the task at hand. To validate that the benefits of EgoPack were not brought by the MTL pre-training alone, we also introduce a *MTL+FT* baseline where a new task-specific head is finetuned for the

	Trained on frozen features	AR		OSCC	LTA		PNR
		Verbs Top-1 (%)	Nouns Top-1 (%)	Acc. (%)	Verbs ED (↓)	Nouns ED (↓)	Loc. Err. (s) (↓)
Ego4D Baselines [25]	✗	22.18	21.55	68.22	0.746	0.789	0.62
EgoT2s [68]	✗	23.04	23.28	**72.69**	0.731	0.769	**0.61**
MLP	✓	24.08	30.45	70.47	0.763	**0.742**	1.76
Temporal Graph	✓	24.25	30.43	71.26	0.754	0.752	**0.61**
Multi-Task Learning	✓	22.05	29.44	71.10	0.740	0.746	0.62
Task Translation†	✓	23.68	28.28	71.48	0.740	0.756	**0.61**
EgoPack	✓	**25.10**	**31.10**	71.83	**0.728**	0.752	**0.61**

Table 1. EgoPack on Ego4D HOI tasks. MLP is a simple baseline consisting of a few linear layers, while Temporal Graph models all tasks using a unified temporal graph-based architecture. MTL [54] uses hard parameter sharing to jointly learn all tasks, which may result in negative transfers. Ego-T2s [68] learns to translate features across tasks to optimise the primary task. EgoPack builds on the unified architecture of the Temporal Graph and learns to exploit the perspective of different tasks for efficient transfers to the novel task. Performances of EgoPack are evaluated over three runs using accuracy for AR and OSCC, Edit Distance for LTA and temporal localisation error for PNR. †Our implementation of the task translation mechanism from EgoT2 [68] using Omnivore features.

	Temp. model	Multi-Task Objective	Cross-Task Interaction	Metrics Average (Δ)
MLP	✗	✗	✗	0.416
Temp.	✓	✗	✗	0.433 (+4.22%)
Task Transl.	✓	✗	✓	0.431 (+3.61%)
MTL	✓	✓	✓	0.430 (+3.50%)
MTL+FT	✓	✓	✓	0.437 (+5.02%)
EgoPack	✓	✗	✓	**0.441 (+6.10%)**

Table 2. Ablation of the different contributions in EgoPack, measured according an aggregated score, computed as the mean of the standardised metrics across tasks.

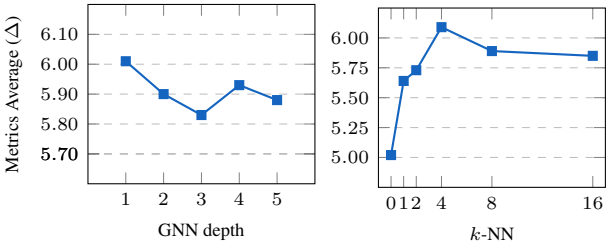


Figure 4. Parameter analysis for the cross-tasks interaction module of EgoPack. We analyse the impact on performance of GNN depth and the number of nearest neighbours denoted as k-NN.

novel task, without access to the output of the other heads. The limited performance of this configuration could be explained by the model losing the knowledge learnt during the multi-task learning, without a significant improvement over the single-task baselines, thus only partially reusing the gained knowledge. On the contrary, EgoPack preserves this knowledge in the form of prototypes, which proves to be effective for retaining the model's knowledge when learning a new task.

Depth of the GNN and the selection of k We observe that EgoPack is quite robust to the number of GNN layers in the interaction stage between the input features and the task prototypes, as shown in Fig. 4. Regarding the selection of the k parameter, we compare the *MTL+FT* baseline ($k = 0$) with EgoPack. The best performance is achieved at $k = 4$ with a saturating trend afterwards, showing that interacting with a limited number of prototypes is sufficient.

Results on the test set We compare EgoPack on the test set of PNR, OSCC and LTA benchmarks, to validate the improvements and soundness of EgoPack. In this setting, a fair comparison between methods is challenging because of the use of different backbones, supervision levels, ensemble strategies and challenge-specific tuning, such as training also on the validation set. Remarkably, we achieve SOTA performances in LTA, outperforming the other methods that finetune the entire backbone, with a more evident benefit in the verbs edit distance. In PNR, we closely match other approaches while the improvement is more limited in the OSCC task. In this task, we notice a relevant impact of the Ego4D pretraining on the performance. We provide a more in-depth description of the differences between these methods in the supplementary materials.

4.3. Qualitative results

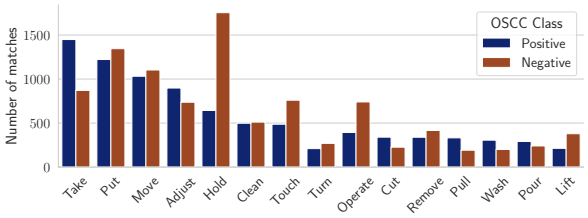
Closest Task Prototypes We evaluate which are the closest task-specific prototypes in Fig. 5. In this example, OSCC is the novel task and the model has access to the prototypes of the learnt tasks, namely LTA, AR and PNR. We focus on the prototypes from the last two tasks and group together nodes that share the same verb label to make the picture more readable. Looking at the number of occurrences of the prototypes, we observe that some nodes are more discriminative to detect a state change, *e.g. peel* and *hold* actions are typically associated (*peel*) or not (*hold*) with state changes, and therefore show more evident peaks for positive and negative classes, indicating the network is using these clues to solve the task.

PNR	Ego4D Pt.	Loc. Error (s) (↓)
CNN LSTM [25]	✗	0.76
EgoVLP [42]	✓	0.67
EgoT2 [68]	✗	0.66
EgoPack	✗	**0.66**

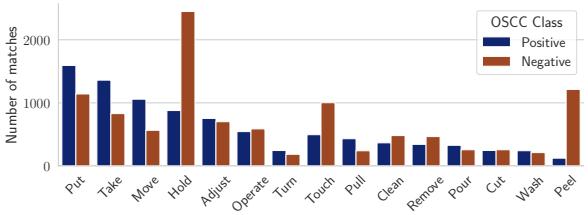
OSCC	Ego4D Pt.	Accuracy (%)
I3D RN-50 [25]	✗	67.6
EgoVLP [42]	✓	74.0
EgoT2 (EgoVLP) [68]	✓	**75.0**
EgoT2 (I3D) [68]	✗	71.0
EgoPack	✗	72.1

LTA	Ego4D Pt.	Verb (↓)	Noun (↓)	Action (↓)
SlowFast [25]	✗	0.739	0.780	0.943
EgoT2 [68]	✗	0.722	0.764	0.935
HierVL [1]	✓	0.724	**0.735**	0.928
I-CVAE [43]	✗	0.741	0.740	0.930
EgoPack	✗	**0.721**	**0.735**	**0.925**

Table 3. Comparison of EgoPack on the test set of the Ego4D benchmarks. For a fair comparison, we differentiate between methods that have been pretrained on full Ego4D (✓) and those that have been trained only on the benchmark data (✗), which includes EgoPack.



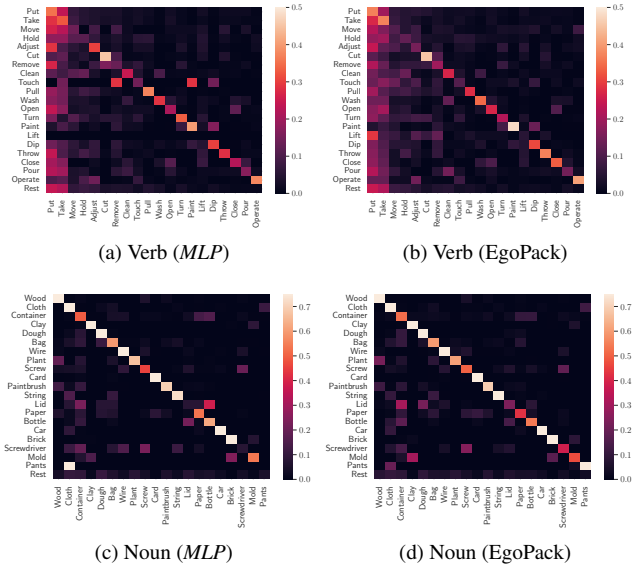
(a) AR Task Prototypes



(b) PNR Task Prototypes

Figure 5. Closest nodes to the OSCC samples among AR and PNR task prototypes. Some nodes appear to be more discriminative of the presence or absence of an object state change.

Confusion matrices In Fig. 6, we compare the confusion matrix of EgoPack with the MLP model for the top-20 largest verb and noun classes in the AR task, grouping the remaining classes in a "*rest*" pseudo-class. Overall, we observe an evident improvement on the noun labels, due to the positive effect of cross-tasks interaction. For example, the network appears to better disambiguate between objects that may appear at the same time in the scene, *e.g.* "*pants*" and "*cloth*" or "*bottle*" and "*lid*", which we speculate to be the result of a better ability of other tasks, namely OSCC, to identify active objects. Regarding the verbs, we also observe notable improvements, in addition to better recognition of verbs that are the temporal inverse of each other, *e.g.* "*put*" and "*take*" or "*open*" and "*close*", thanks to the improved temporal reasoning of our unified model.



(a) Verb (*MLP*) (b) Verb (EgoPack)

(c) Noun (*MLP*) (d) Noun (EgoPack)

Figure 6. Action Recognition confusion matrix of EgoPack compared to the MLP *baseline* for the top-20 verb and noun classes.

5. Conclusions and future work

We presented EgoPack, a framework that allows knowledge sharing between different egocentric vision tasks, enabling an efficient use of the perspectives that each task can provide. We built EgoPack on top of a unified temporal architecture that can model distinct tasks with a shared backbone and minimal task-specific overhead. EgoPack overcomes the main limitation posed by traditional multi-task learning approaches, namely the unrealistic expectation that supervision is available for all tasks at training time. Indeed, the prototypes mechanism behind EgoPack allows to create a summary of what the model has learnt so far as it abstracts the task-specific knowledge that could be used in novel tasks. The model can then be updated to the any new task, while also peeking at the perspective of the previous tasks. Results on Ego4D validate our approach, showing competitive performance with other methods, even without end to end training.

Acknowledgements

This study was carried out within the FAIR - Future Artificial Intelligence Research and received funding from the European Union Next-GenerationEU (PIANO NAZIONALE DI RIPRESA E RESILIENZA (PNRR) – MISSIONE 4 COMPONENTE 2, INVESTIMENTO 1.3 – D.D. 1555 11/10/2022, PE00000013). This manuscript reflects only the authors' views and opinions, neither the Eu-

ropean Union nor the European Commission can be considered responsible for them. We acknowledge the CINECA award under the ISCRA initiative, for the availability of high performance computing resources and support.

References

[1] Kumar Ashutosh, Rohit Girdhar, Lorenzo Torresani, and Kristen Grauman. Hiervl: Learning hierarchical video-language embeddings. In *CVPR*, 2023. 2, 8, 12, 13

[2] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016. 4

[3] Alejandro Betancourt, Pietro Morerio, Carlo S Regazzoni, and Matthias Rauterberg. The evolution of first person vision methods: A survey. *IEEE TCSVT*, 2015. 2

[4] Joao Carreira and Andrew Zisserman. Quo vadis, action recognition? a new model and the kinetics dataset. In *CVPR*, 2017. 5, 13

[5] Rich Caruana. Multitask learning. *Machine learning*, 28: 41–75, 1997. 3

[6] Min-Hung Chen, Zsolt Kira, Ghassan AlRegib, Jaekwon Yoo, Ruxin Chen, and Jian Zheng. Temporal attentive alignment for large-scale video domain adaptation. In *ICCV*, 2019. 2

[7] Ting Chen, Saurabh Saxena, Lala Li, Tsung-Yi Lin, David J Fleet, and Geoffrey E Hinton. A unified sequence interface for vision tasks. In *NeurIPS*, 2022. 3

[8] Tianlong Chen, Xuxi Chen, Xianzhi Du, Abdullah Rashwan, Fan Yang, Huizhong Chen, Zhangyang Wang, and Yeqing Li. Adamv-moe: Adaptive multi-task vision mixture-of-experts. In *ICCV*, 2023. 3

[9] Zhao Chen, Vijay Badrinarayanan, Chen-Yu Lee, and Andrew Rabinovich. Gradnorm: Gradient normalization for adaptive loss balancing in deep multitask networks. In *ICML*, 2018. 3, 13

[10] Yuanzheng Ci, Yizhou Wang, Meilin Chen, Shixiang Tang, Lei Bai, Feng Zhu, Rui Zhao, Fengwei Yu, Donglian Qi, and Wanli Ouyang. Unihcp: A unified model for human-centric perceptions. In *CVPR*, 2023. 3

[11] Dima Damen, Hazel Doughty, Giovanni Maria Farinella, Sanja Fidler, Antonino Furnari, Evangelos Kazakos, Davide Moltisanti, Jonathan Munro, Toby Perrett, Will Price, and Michael Wray. The epic-kitchens dataset: Collection, challenges and baselines. *IEEE TPAMI*, 2021. 2

[12] Dima Damen, Hazel Doughty, Giovanni Maria Farinella, Antonino Furnari, Jian Ma, Evangelos Kazakos, Davide Moltisanti, Jonathan Munro, Toby Perrett, Will Price, and Michael Wray. Rescaling egocentric vision: Collection, pipeline and challenges for epic-kitchens-100. *IJCV*, 2022. 2

[13] Eadom Dessalene, Michael Maynord, Chinmaya Devaraj, Cornelia Fermuller, and Yiannis Aloimonos. Egocentric object manipulation graphs. *arXiv preprint arXiv:2006.03201*, 2020. 3

[14] Eadom Dessalene, Chinmaya Devaraj, Michael Maynord, Cornelia Fermuller, and Yiannis Aloimonos. Forecasting action through contact representations from first person video. *IEEE TPAMI*, 2021. 3

[15] Wenqi Fan, Yao Ma, Qing Li, Yuan He, Eric Zhao, Jiliang Tang, and Dawei Yin. Graph neural networks for social recommendation. In *The world wide web conference*, 2019. 2

[16] Christoph Feichtenhofer, Haoqi Fan, Jitendra Malik, and Kaiming He. Slowfast networks for video recognition. In *ICCV*, 2019. 3, 14

[17] Chris Fifty, Ehsan Amid, Zhe Zhao, Tianhe Yu, Rohan Anil, and Chelsea Finn. Efficiently identifying task groupings for multi-task learning. In *NeurIPS*, 2021. 3

[18] Antonino Furnari and Giovanni Maria Farinella. Rolling-unrolling lstms for action anticipation from first-person video. *IEEE TPAMI*, 2020. 2

[19] Antonino Furnari, Sebastiano Battiato, Kristen Grauman, and Giovanni Maria Farinella. Next-active-object prediction from egocentric videos. *Journal of Visual Communication and Image Representation*, 2017. 2

[20] Ruohan Gao, Tae-Hyun Oh, Kristen Grauman, and Lorenzo Torresani. Listen to look: Action recognition by previewing audio. In *CVPR*, 2020. 2

[21] Pallabi Ghosh, Nirat Saini, Larry S Davis, and Abhinav Shrivastava. All about knowledge graphs for actions. *arXiv preprint arXiv:2008.12432*, 2020. 3

[22] Pallabi Ghosh, Yi Yao, Larry Davis, and Ajay Divakaran. Stacked spatio-temporal graph convolutional networks for action segmentation. In *WACV*, 2020. 3

[23] Rohit Girdhar and Kristen Grauman. Anticipative video transformer. In *ICCV*, 2021. 2

[24] Rohit Girdhar, Mannat Singh, Nikhila Ravi, Laurens van der Maaten, Armand Joulin, and Ishan Misra. Omnivore: A single model for many visual modalities. In *CVPR*, 2022. 3, 5, 13

[25] Kristen Grauman, Andrew Westbury, Eugene Byrne, Zachary Chavis, Antonino Furnari, Rohit Girdhar, Jackson Hamburger, Hao Jiang, Miao Liu, Xingyu Liu, et al. Ego4d: Around the world in 3,000 hours of egocentric video. In *CVPR*, 2022. 2, 5, 7, 8, 11, 12, 14

[26] Jiuxiang Gu, Zhenhua Wang, Jason Kuen, Lianyang Ma, Amir Shahroudy, Bing Shuai, Ting Liu, Xingxing Wang, Gang Wang, Jianfei Cai, et al. Recent advances in convolutional neural networks. *PR*, 2018. 2

[27] Michelle Guo, Albert Haque, De-An Huang, Serena Yeung, and Li Fei-Fei. Dynamic task prioritization for multitask learning. In *ECCV*, 2018. 3, 13

[28] Pengsheng Guo, Chen-Yu Lee, and Daniel Ulbricht. Learning to branch for multi-task learning. In *ICML*, 2020. 3

[29] Will Hamilton, Zhitao Ying, and Jure Leskovec. Inductive representation learning on large graphs. In *NeurIPS*, 2017. 4

[30] Dan Witzner Hansen and Qiang Ji. In the eye of the beholder: A survey of models for eyes and gaze. *IEEE TPAMI*, 2009. 2

[31] Thomas E. Huang, Yifan Liu, Luc Van Gool, and Fisher Yu. Video task decathlon: Unifying image and video tasks in autonomous driving. In *ICCV*, 2023. 3

[32] Yifei Huang, Minjie Cai, Zhenqiang Li, Feng Lu, and Yoichi Sato. Mutual context network for jointly estimating egocentric gaze and action. *IEEE TIP*, 2020. 3

[33] Yifei Huang, Yusuke Sugano, and Yoichi Sato. Improving action segmentation via graph-based temporal reasoning. In *CVPR*, 2020. 2, 3

[34] Youngkyoon Jang, Brian Sullivan, Casimir Ludwig, Iain Gilchrist, Dima Damen, and Walterio Mayol-Cuevas. Epic-tent: An egocentric video dataset for camping tent assembly. In *ICCVW*, 2019. 2

[35] Zhuoliang Kang, Kristen Grauman, and Fei Sha. Learning with whom to share in multi-task feature learning. In *ICML*, 2011. 3

[36] Georgios Kapidis, Ronald Poppe, Elsbeth van Dam, Lucas Noldus, and Remco Veltkamp. Multitask learning to improve egocentric action recognition. In *ICCVW*, 2019. 3

[37] Steven Kearnes, Kevin McCloskey, Marc Berndl, Vijay Pande, and Patrick Riley. Molecular graph convolutions: moving beyond fingerprints. *Journal of computer-aided molecular design*, 2016. 2

[38] Alex Kendall, Yarin Gal, and Roberto Cipolla. Multi-task learning using uncertainty to weigh losses for scene geometry and semantics. In *CVPR*, 2018. 3, 13

[39] Asifullah Khan, Anabia Sohail, Umme Zahoora, and Aqsa Saeed Qureshi. A survey of the recent architectures of deep convolutional neural networks. *Artificial intelligence review*, 2020. 2

[40] Iasonas Kokkinos. Ubernet: Training a universal convolutional neural network for low-, mid-, and high-level vision using diverse datasets and limited memory. In *CVPR*, 2017. 2, 3, 6

[41] Zewen Li, Fan Liu, Wenjie Yang, Shouheng Peng, and Jun Zhou. A survey of convolutional neural networks: analysis, applications, and prospects. *IEEE transactions on neural networks and learning systems*, 2021. 2

[42] Kevin Qinghong Lin, Jinpeng Wang, Mattia Soldan, Michael Wray, Rui Yan, Eric Z XU, Difei Gao, Rong-Cheng Tu, Wenzhe Zhao, Weijie Kong, et al. Egocentric video-language pretraining. In *NeurIPS*, 2022. 8, 12, 13

[43] Esteve Valls Mascaró, Hyemin Ahn, and Dongheui Lee. Intention-conditioned long-term human egocentric action anticipation. In *WACV*, 2023. 8, 12, 13, 14

[44] Jonathan Munro and Dima Damen. Multi-modal domain adaptation for fine-grained action recognition. In *CVPR*, 2020. 2

[45] Tushar Nagarajan, Yanghao Li, Christoph Feichtenhofer, and Kristen Grauman. Ego-topo: Environment affordances from egocentric video. In *CVPR*, 2020. 3

[46] Adrián Núñez-Marcos, Gorka Azkune, and Ignacio Arganda-Carreras. Egocentric vision-based action recognition: A survey. *Neurocomputing*, 2022. 2

[47] Francesca Pistilli and Giuseppe Averta. Graph learning in robotics: a survey. *IEEE Access*, 2023. 2

[48] Mirco Planamente, Chiara Plizzari, Simone Alberto Peirone, Barbara Caputo, and Andrea Bottino. Relative norm alignment for tackling domain shift in deep multi-modal classification. *IJCV*, 2024. 2

[49] Chiara Plizzari, Gabriele Goletto, Antonino Furnari, Siddhant Bansal, Francesco Ragusa, Giovanni Maria Farinella, Dima Damen, and Tatiana Tommasi. An outlook into the future of egocentric vision. *arXiv preprint arXiv:2308.07123*, 2023. 2

[50] Chiara Plizzari, Toby Perrett, Barbara Caputo, and Dima Damen. What can a cook in italy teach a mechanic in india? action recognition generalisation over scenarios and locations. In *ICCV*, 2023. 2, 5

[51] Shraman Pramanick, Yale Song, Sayan Nag, Kevin Qinghong Lin, Hardik Shah, Mike Zheng Shou, Rama Chellappa, and Pengchuan Zhang. Egovlpv2: Egocentric video-language pre-training with fusion in the backbone. In *ICCV*, 2023. 2

[52] Santhosh K. Ramakrishnan, Ziad Al-Halah, and Kristen Grauman. Spotem: Efficient video search for episodic memory. In *ICLR*, 2023. 2

[53] Maheen Rashid, Hedvig Kjellstrom, and Yong Jae Lee. Action graphs: Weakly-supervised action localization with graph convolution networks. In *WACV*, 2020. 3

[54] Sebastian Ruder. An overview of multi-task learning in deep neural networks. *arXiv preprint arXiv:1706.05098*, 2017. 3, 7

[55] Alvaro Sanchez-Gonzalez, Jonathan Godwin, Tobias Pfaff, Rex Ying, Jure Leskovec, and Peter Battaglia. Learning to simulate complex physics with graph networks. In *ICML*, 2020. 2

[56] Fadime Sener, Dibyadip Chatterjee, Daniel Shelepov, Kun He, Dipika Singhania, Robert Wang, and Angela Yao. Assembly101: A large-scale multi-view video dataset for understanding procedural activities. In *CVPR*, 2022. 2

[57] Haosen Shi, Shen Ren, Tianwei Zhang, and Sinno Jialin Pan. Deep multitask learning with progressive parameter sharing. In *ICCV*, 2023. 3

[58] Martin Simonovsky and Nikos Komodakis. Dynamic edge-conditioned filters in convolutional neural networks on graphs. In *CVPR*, 2017. 2

[59] Ayan Sinha, Zhao Chen, Vijay Badrinarayanan, and Andrew Rabinovich. Gradient adversarial training of neural networks. *arXiv preprint arXiv:1806.08028*, 2018. 3, 13

[60] Trevor Standley, Amir Zamir, Dawn Chen, Leonidas Guibas, Jitendra Malik, and Silvio Savarese. Which tasks should be learned together in multi-task learning? In *ICML*, 2020. 3

[61] Ximeng Sun, Rameswar Panda, Rogerio Feris, and Kate Saenko. Adashare: Learning what to share for efficient deep multi-task learning. In *NeurIPS*, 2020. 3

[62] Zehua Sun, Qiuhong Ke, Hossein Rahmani, Mohammed Bennamoun, Gang Wang, and Jun Liu. Human action recognition from various data modalities: A review. *IEEE TPAMI*, 2023. 2

[63] Simon Vandenhende, Stamatios Georgoulis, and Luc Van Gool. Mti-net: Multi-scale task interaction networks for multi-task learning. In *ECCV*, 2020. 3, 13

[64] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *NeurIPS*, 2017. 3

[65] Xiaohan Wang, Linchao Zhu, Heng Wang, and Yi Yang. Interactive prototype learning for egocentric action recognition. In *ICCV*, 2021. 3

[66] Yue Wang, Yongbin Sun, Ziwei Liu, Sanjay E Sarma, Michael M Bronstein, and Justin M Solomon. Dynamic graph cnn for learning on point clouds. *ACM Transactions on Graphics*, 2019. 2

[67] Sen Wu, Hongyang R. Zhang, and Christopher Ré. Understanding and improving information transfer in multi-task learning. In *ICLR*, 2020. 6

[68] Zihui Xue, Yale Song, Kristen Grauman, and Lorenzo Torresani. Egocentric video task translation. In *CVPR*, 2023. 2, 3, 5, 6, 7, 8, 12

[69] Shen Yan, Xuehan Xiong, Anurag Arnab, Zhichao Lu, Mi Zhang, Chen Sun, and Cordelia Schmid. Multiview transformers for video recognition. In *CVPR*, 2022. 1

[70] Lijin Yang, Yifei Huang, Yusuke Sugano, and Yoichi Sato. Interact before align: Leveraging cross-modal knowledge for domain adaptive action recognition. In *CVPR*, 2022. 2

[71] Tianhe Yu, Saurabh Kumar, Abhishek Gupta, Sergey Levine, Karol Hausman, and Chelsea Finn. Gradient surgery for multi-task learning. In *NeurIPS*, 2020. 3, 12, 13

[72] Runhao Zeng, Wenbing Huang, Mingkui Tan, Yu Rong, Peilin Zhao, Junzhou Huang, and Chuang Gan. Graph convolutional networks for temporal action localization. In *ICCV*, 2019. 3

[73] Chen-Lin Zhang, Jianxin Wu, and Yin Li. Actionformer: Localizing moments of actions with transformers. In *ECCV*, 2022. 1, 2

[74] Yu Zhang and Qiang Yang. A survey on multi-task learning. *IEEE Transactions on Knowledge and Data Engineering*, 34 (12):5586–5609, 2021. 3

[75] Yue Zhao, Ishan Misra, Philipp Krähenbühl, and Rohit Girdhar. Learning video representations from large language models. In *CVPR*, 2023. 2

[76] Zeyun Zhong, David Schneider, Michael Voit, Rainer Stiefelhagen, and Jürgen Beyerer. Anticipative feature fusion transformer for multi-modal action anticipation. In *WACV*, 2023. 1, 2

[77] Bolei Zhou, Alex Andonian, Aude Oliva, and Antonio Torralba. Temporal relational reasoning in videos. In *ECCV*, 2018. 5

Appendix

In Appendix A we provide additional details on how the different tasks of Ego4D [25] are modelled as a temporal graph, expanding Sec. 4.1 of the main paper. Appendix B provides additional implementation details of the *Task Translation* model and how the EgoT2 architecture was adapted to our scenario. Appendix C presents additional experiments to evaluate the role of negative transfer in MTL. A more in-depth comparison of the methods on the test-set of the Ego4D challenges is shown in Appendix D. Finally, we show more qualitative results in Appendix E.

A. Additional implementation details

A.1. Temporal modelling

Action Recognition (AR) Action annotations are derived from the LTA benchmark. Each action is an annotated segment lasting approx. 8.0 seconds, and actions may temporally overlap. To provide each action with additional temporal context from the surrounding, we process actions in fixed length sequences of $w = 9$ actions, each mapped to a node in the temporal graph. The target action is the central node of the sequence and the classification loss is computed only on this node. The window size was selected to be larger than the receptive field of the temporal GNN after 3 layers of graph convolution, which we observed to be the optimal depth of the network. Furthermore, using fixed size sequences allows to train the model on videos containing a variable and possibly large number of actions.

Long Term Anticipation (LTA) LTA is formulated as an action anticipation task in which the model is shown $N = 2$ input clips and has to predict the actions occurring in the following $Z = 20$ timestamps. As in AR, clips lasts approx. 8.0 seconds and may overlap. Therefore, the effective temporal window seen by the model may vary between 8.0 and 16.0 seconds, depending on how much the input clips overlap. Input clips and future timestamps are mapped to nodes in the graph, with the latter initialised with the mean of the features of the input clips. Edges connect each node to its subsequent and preceding nodes. Additionally, nodes that represent future actions to be predicted are also connected to the input clips. This connectivity pattern allows local temporal reasoning, *e.g.* to rearrange the order of actions in the anticipation window, while using the global context provided by the input clips to guide the prediction. Similarly to AR, there is a *one-to-one* correspondence between actions and nodes of the graph.

Object State Change Classification (OSCC) and Point of No Return (PNR) Unlike action-based annotations, OSCC and PNR do not necessarily match the boundaries of an action segment of the video. Each segment lasts approx. 8.0 seconds and is uniformly divided in 4 (OSCC) or 16 (PNR) smaller sub-segments that are mapped to the nodes of the graph.

A.1.1 Temporal model sharing across tasks

A key premise of EgoPack is that different tasks are modelled using the same shared temporal backbone architecture, even though the temporal granularity of the different tasks may vary. To achieve this, we do not constraint nodes to represent the same fixed size temporal window across all

	AR		OSCC	LTA		PNR
	Verbs Top-1 (%)	**Nouns Top-1 (%)**	**Acc. (%)**	**Verbs ED (↓)**	**Nouns ED (↓)**	**Loc. Err. (s) (↓)**
Temporal Graph	**24.25**	**30.43**	**71.26**	0.754	0.752	**0.61**
Multi-Task Learning	22.16	29.34	70.93	0.740	**0.746**	0.62
Multi-Task Learning (+ PCGrad [71])	22.01	29.46	70.86	**0.737**	**0.746**	0.63

Table 4. Results of PCGrad [71] compared to vanilla Multi-Task Learning.

PNR	**Pre-trained on Ego4D [25]**	**Trained on pre-extracted features**	**Loc. Error (s) (↓)**
CNN LSTM [25]	✗	✗	0.76
EgoVLP [42]	✓	✗	0.67
EgoT2 [68]	✗	✗	**0.66**
EgoPack	✗	✓	**0.66**

OSCC	**Pre-trained on Ego4D [25]**	**Trained on pre-extracted features**	**Accuracy (%)**
I3D RN-50 [25]	✗	✗	67.6
EgoVLP [42]	✓	✗	74.0
EgoT2 (EgoVLP) [68]	✓	✗	**75.0**
EgoT2 (I3D) [68]	✗	✗	71.0
EgoPack (SlowFast)	✗	✓	72.1

LTA	**Pre-trained on Ego4D [25]**	**Trained on pre-extracted features**	**Verb (↓)**	**Noun (↓)**	**Action (↓)**
SlowFast [25]	✗	✗	0.739	0.780	0.943
EgoT2 [68]	✗	✗	0.722	0.764	0.935
HierVL [1]	✓	✗	0.724	**0.735**	0.928
I-CVAE [43]	✗	✓	0.741	0.740	0.930
EgoPack	✗	✓	**0.721**	**0.735**	**0.925**

Table 5. Comparison of EgoPack on the test set of the Ego4D benchmarks, highlighting differences in terms of additional Ego4D pretraining and use of pre-extracted features.

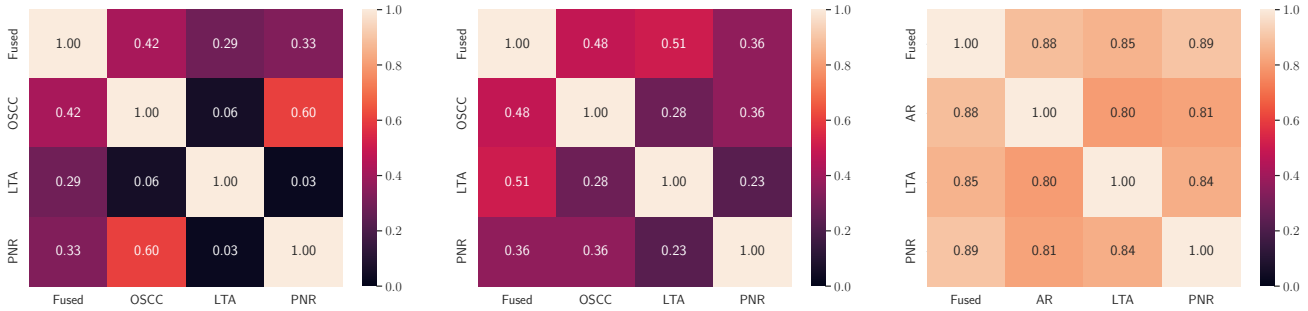
tasks. Through the utilisation of a multi-task learning process, we force the network to jointly learn tasks with different temporal resolutions, enabling reasoning at different temporal scales. This formulation is particularly effective to prepare the model to new tasks, as the model has already learnt to combine tasks with potentially different temporal resolutions during the MTL training. As an example, consider the case in which the MTL model is trained on AR, LTA, OSCC and PNR. In this case, the nodes of the temporal graph represent actions when the task is AR or LTA, or shorter temporal sub-segments for OSCC and PNR. To train EgoPack, we update the weights of $\mathcal{M}_t$ for all novel tasks, except LTA for which we observe better performance by not updating the temporal model.

B. Task Translation Implementation Details

The objective of the *Task Translation* experiments is to compare the task translation mechanism proposed by EgoT2 [68], which learns a mapping between features extracted from different task-specific models, to EgoPack which leverages past gained knowledge under the form of task-specific prototypes. For fair comparison, we re-implement this mechanism and evaluate it on top of the same temporal backbone and the same pre-extracted features of EgoPack. We start from the EgoT2-g model and employ the same architecture for the *Task Translation*, which consists of a 1-layer encoder-decoder stack, each with 8 heads, dropout 0.1 and features size 1024. The input of the *Task Translation* is provided by an ensemble of *Temporal Graph* models, one for each task. The whole architecture is trained for one task at the time, as EgoPack, and only the encoder-decoder architecture is updated, while the temporal models that compose the ensemble are kept frozen. We train *Task Translation* for 30 epochs, using the Adam optimiser with learning rate 1×10^{-4} (with the exception of OSCC which uses learning rate 1×10^{-3}), batch size 16, linear warmup for the first 5 epochs and weight decay 1×10^{-5}.

C. Additional Multi-Task Experiments

MTL suffers from negative transfers between different tasks, and fine-tuning an MTL on a new task may not be the most effective approach to retain knowledge learned in



(a) AR Verb (b) AR Noun (c) OSCC

Figure 7. Agreement ratio between predictions from different tasks when the novel task is Action Recognition (Fig. 7a and Fig. 7b) and Object State Change Classification (Fig. 7c). *Fused* represents the sum of the logits from the auxiliary tasks.

	AR		**OSCC**	**LTA**		**PNR**
	Verbs Top-1 (%)	**Nouns Top-1 (%)**	**Acc. (%)**	**Verbs ED (↓)**	**Nouns ED (↓)**	**Loc. Err. (s) (↓)**
MTL (All tasks)	22.05	29.44	71.10	0.740	**0.746**	0.62
MTL (3 tasks) + FT	24.36	**31.31**	71.60	0.744	0.754	0.62
MTL (3 tasks) + TT	22.30	29.50	70.96	0.738	0.757	0.62
EgoPack	**25.10**	31.10	**71.83**	**0.728**	0.752	**0.61**

Table 6. Comparison of vanilla MTL and two finetuning strategies to extend MTL models to novel tasks.

the MTL training process. We observe evidence of this phenomenon in Table 6, where we compare the MTL on all tasks with two finetuning approaches to extend a model trained on three tasks to a fourth novel task. MTL+FT finetunes the model for the novel task, as already shown in Table 2, while MTL+TT replaces the EgoPack's second stage with a decoder analogous to TT, which learns the new task as a "recombination" of the previous tasks.

Brute Force Multi-Task Learning Table 7 presents a comprehensive analysis of MTL on all task combinations, to assess the effect of negative transfer when a smaller subset of tasks is used. Even with fewer tasks, MTL still suffers from negative transfer across tasks and does not represent an upper bound for EgoPack, which is showing a clear advantage.

Minimising negative transfer Various approaches have been proposed to address the issue of negative transfer in multi-task learning [9, 27, 38, 59, 63, 71]. Although the multi-task setting significantly differs from the settings proposed for EgoPack, we provide a comparison with one of these methods, PCGrad [71], which projects tasks' gradients on the normal plane of all the other gradients to remove interference among tasks. Apart from minimal fluctuations, PCGrad does not appear to significantly improve over MTL, showing that these methods may still be insufficient to effectively reduce the negative transfer, as shown in Table 4.

D. Comparison of methods on the test-set

We summarise the main differences between EgoPack and the other methods on the test-set in Table 5, extending Table 3 of the main paper and highlighting differences in terms of additional Ego4D pretraining and use of pre-extracted features. EgoPack relies on pre-extracted features from Omnivore [24], which was trained on Kinetics-400 [4] for action recognition. As a result, these features are highly semantic and may struggle to encode finer temporal details required by certain tasks, *e.g.* to detect changes in the objects being manipulated in OSCC or PNR. Most other methods, with the exception of I-CVAE [43], train also their features extraction backbones on Ego4D benchmarks' data, which allows to learn task-specific models more suited for the task at hand. On the contrary, we do not update the features extraction backbone when training EgoPack.

When evaluating EgoPack on the test-set, we also observe a significant performance gap compared to other methods that rely on some amount of additional data from Ego4D, while the benchmarks data are more limited in size. HierVL [1] is pretrained on the full Ego4D using a contrastive video-language objective with short-term and long-term textual narrations. EgoVLP [42] is pretrained on a large subset of Ego4D using a video-language contrastive objective with action-aware positive samples and scene-aware negative samples. The only method directly comparable to EgoPack in terms of pre-training data and parame-

				AR		OSCC	LTA		PNR
AR	**OSCC**	**LTA**	**PNR**	**Verbs Top-1 (%)**	**Nouns Top-1 (%)**	**Acc. (%)**	**Verbs ED (↓)**	**Nouns ED (↓)**	**Loc. Err. (s) (↓)**
Single tasks				24.25	30.43	71.26	0.754	0.752	**0.61**
✓	✓	-	-	23.98	30.60	70.81	-	-	-
✓	-	✓	-	22.23	29.48	-	0.744	0.744	-
✓	-	-	✓	24.05	30.72	-	-	-	0.63
-	✓	✓	-	-	-	70.71	0.745	0.751	-
-	✓	-	✓	-	-	71.01	-	-	0.66
-	-	✓	✓	-	-	-	0.751	0.752	0.62
✓	✓	✓	-	22.05	29.44	71.10	0.739	0.745	-
✓	✓	-	✓	23.82	30.83	71.03	-	-	0.63
✓	-	✓	✓	22.24	29.83	-	0.745	**0.743**	0.62
-	✓	✓	✓	-	-	71.06	0.746	0.751	0.63
MTL (All tasks)				22.05	29.44	71.10	0.740	0.746	0.62
EgoPack				**25.10**	**31.10**	**71.83**	**0.728**	0.752	**0.61**

Table 7. Brute force experiments in multi-task learning with all combinations of tasks.

ters updated is I-CVAE [43], which uses the SlowFast [16] features released by [25] for the LTA benchmark. The extension of EgoPack to additional backbones, possibly with end-to-end finetuning, is outside of the scope of this paper and is left as a future work. For OSCC, we report the results of EgoPack using SlowFast features instead of Omnivore as they showed better performances compared to the latter.

E. Additional qualitative results

EgoPack fuses the predictions coming from different task perspectives by summing the task-specific logits. We show in Fig. 7 the agreement ratio between the predictions produced by the different tasks $\mathbf{y}_i^k$ and the final output computed as the sum of the individual contributions $\mathbf{y}_i = \sum_k \mathbf{y}_i^k$. In Action Recognition, we observe low agreement both between task pairs and with respect to the fused predictions, suggesting that they contribute complementary information to the novel task. On the other hand, in OSCC, tasks predictions tend to be more consistent across tasks.